

x402 on Algorand: A Production Integration Reference for the PYTHAI/DELTAVERSE/BANKON Ecosystem

TL;DR

- The x402 Algorand stack is production-ready today: the `exact` scheme on Algorand (the "x402-avm" scheme) settles USDC ASA `31566704` over native atomic transaction groups with instant block-level finality (Algorand confirms blocks every ~2.82 seconds on average), and GoPlausible operates a live multichain facilitator at `https://facilitator.goplausible.xyz` whose mainnet fee-payer is `ZMFK20I7ZBD2U27ISERZC4S6LKM6WMFJPZQ4MYNJDZ2VNBNMBA67RA22AA`. Ship mindX's paid API on Algorand mainnet using the GoPlausible facilitator, wrap the client side with Parsec, and you have a working agentic-commerce rail.
- Two parallel npm scopes exist: the upstream `@x402/*` (x402-foundation) and GoPlausible's Algorand-focused `@x402-avm/*` line; both expose `core`, `avm`, `express`, `hono`, `next`, `fetch`, `axios`, `paywall`, and `extensions` packages, plus a Python `x402-avm` SDK. As of v2.6 these no longer depend on `algosdk` directly (they use `@algorandfoundation/algokit-utils` v10 alpha), which matters for your pure-`node:*` Parsec module.
- The Global x402 Challenge requires a paid x402 endpoint live on Algorand mainnet routing through the GoPlausible facilitator, with a \$100K USD + 500K ALGO prize pool and finals at Devcon 8 India. This aligns with—but is secondary to—your production goal; building mindX/AgenticPlace endpoints on mainnet through GoPlausible simultaneously qualifies you for the leaderboard. (Note a date discrepancy flagged in Caveats: the Official Rules PDF gives a June 12–September 1, 2026 registration window, while Algorand's public pages describe a vaguer "mid-July opening, October measurement" timeline—verify against the live rules before relying on specific dates.)

Key Findings

The complete developer-resource map (every link from the developers page)

The canonical entry point is `https://algorand.co/agentic-commerce/x402/developers`. Every outbound technical link, followed to its destination:

Core scheme + mechanism

- AVM mechanism npm package: `https://www.npmjs.com/package/@x402/avm`
- AVM mechanism source (upstream monorepo): `https://github.com/x402-foundation/x402/tree/main/typescript/packages/mechanisms/avm`

- Algorand **exact** scheme specification: (https://github.com/x402-foundation/x402/blob/main/specs/schemes/exact/scheme_exact_algo.md) (mirrored at (https://github.com/coinbase/x402/blob/main/specs/schemes/exact/scheme_exact_algo.md), merged via Coinbase PR #361)

Facilitator (GoPlausible)

- Facilitator root: (<https://facilitator.goplausible.xyz>)
- OpenAPI docs: (<https://facilitator.goplausible.xyz/docs>)
- Supported networks (live JSON): (<https://facilitator.goplausible.xyz/supported>)
- Live protected demo endpoint: (<https://x402.goplausible.xyz/protected>)
- Live demo playground: (<https://x402.goplausible.xyz/>)

Server middleware

- ExpressJS: (<https://www.npmjs.com/package/@x402/express>)
- Hono: (<https://www.npmjs.com/package/@x402/hono>), example at (<https://github.com/algorandfoundation/x402-demo/tree/main/x402-examples/server/hono>)

Clients

- Fetch: (<https://www.npmjs.com/package/@x402/fetch>), example at (<https://github.com/algorandfoundation/x402-demo/tree/main/x402-examples/client/fetch>)
- Axios: (<https://www.npmjs.com/package/@x402/axios>), example at (<https://github.com/algorandfoundation/x402-demo/tree/main/x402-examples/client/axios>)

Bazaar extension

- Extensions package (GoPlausible scope): (<https://www.npmjs.com/package/@x402-avm/extensions>)
- Examples: (<https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/typescript/x402-avm-extensions-examples.md>)

Official reference + engagement

- Reference implementation docs: (<https://dev.algorand.co/resources/x402-on-algorand/>)
- Workshops: (<https://algorand.co/developers/workshops>)

- Agentic x402 Skills: <https://github.com/algorand-devrel/algorand-agent-skills>
- Blog (conceptual): <https://algorand.co/blog/x402-unlocking-the-agentic-commerce-era>
- Partner form: <https://fjtzqz.share-eu1.hsforms.com/2fAL6Z0blQnCB1YzXtGLLSA>

GoPlausible master documentation (the single richest resource, not directly linked from the developers page but canonical):

<https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/README.md>. Per-package example pages branch from it for both TypeScript (`core`, `avm`, `express`, `hono`, `next`, `fetch`, `axios`, `extensions`, `paywall`) and Python (`avm`, `fastapi`, `flask`, `httpx`, `requests`, `extensions`), plus a legacy V1 (`x402-avm`, `x402-avm-express`, etc.) and a V2-AlgoSDK-legacy directory. The Python SDK is published at <https://pypi.org/project/x402-avm/>.

The x402-avm `exact` scheme: how Algorand differs from EVM

Unlike the EVM scheme (which relies on EIP-3009 `transferWithAuthorization` signatures against an ERC-20 contract address), the Algorand `exact` scheme uses **native ASAs—no smart contract required**. The payment is an atomic transaction group; the `asset` field is a 64-bit ASA ID string (e.g. `"31566704"`) rather than a contract address, and the resource server **MUST** validate it. The facilitator can only direct funds to the `payTo` address specified by the resource server—it has no custody power, which is the scheme's core security property.

Key constants (from the GoPlausible constants reference): USDC mainnet ASA `31566704` (6 decimals); USDC testnet ASA `10458941`; min fee `1000` microAlgos; max atomic group size `16` top-level transactions (256 inner); max reasonable fee-payer fee `10,000,000` microAlgos (10 ALGO safety cap). Mainnet CAIP-2

`algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`; testnet CAIP-2 `algorand:SG01GKSzyE7IEPItTxCBYw9x8FmnrCDexi9/c0UJ0iI=`. The string after `algorand:` is the network genesis hash (CAIP-2 per ChainAgnostic). V1 legacy identifiers (`algorand-mainnet`, `algorand-testnet`) remain supported via automatic mapping. Signatures on top-level transactions may be Ed25519 single-sig (`sig`), k-of-n multisig (`msig`), or logic signature (`lsig`); transactions are msgpack-encoded (base64 per group element). Encoding uses the `algorand-msgpack` library.

Payment flow (server/facilitator/client)

1. Client requests protected resource; server returns `402` + `paymentRequirements` (scheme, network, amount, asset, payTo, maxTimeoutSeconds, extra).
2. Client builds an atomic group. **Without fee abstraction:** a single `axfer` (ASA transfer) signed by the client with note `"x402-payment-v2"`. **With fee abstraction** (`feePayer` present in `extra`): a 2-transaction group—index 0 is an unsigned `pay` self-payment from

- the facilitator's fee-payer with pooled fee ($\text{minFee} \times 2$), FlatFee true, note ("x402-fee-payer")); index 1 is the client-signed (axfer) with fee 0. (paymentIndex) identifies the paying transaction (MUST be 0 for a standalone transaction).
3. Client signs only its own transaction, base64-msgpack-encodes the group into the (paymentGroup) array, sends the request with the (X-PAYMENT) header.
 4. Resource server forwards ({ x402Version, paymentPayload, paymentRequirements }) to the facilitator's (POST /verify).
 5. Facilitator validates (group ≤ 16; decodes all txns; only allows unsigned txns from facilitator-managed addresses; checks group-ID consistency; rejects (keyreg), unbalanced (rekeyTo), and any (closeRemainderTo)/(assetCloseTo) to prevent account draining; verifies the (axfer) at (paymentIndex) matches asset/payTo/amount and is signed; verifies the fee-payer (pay) txn has amount 0, sender in managed addresses, fee ≤ 10 ALGO), signs the fee-payer txn, and **simulates** the full group against an Algorand node's (simulate) endpoint, returning ({ isValid, invalidReason? }).
 6. On valid, server calls (POST /settle); facilitator re-verifies, signs facilitator-managed transactions, submits the group via (v2/transactions), extracts the payment txId, and returns ({ success, transaction (txId), network }).
 7. Instant finality—Algorand has no consensus forks, so as soon as the txn is in a block the payment is settled; server returns (200) + resource and a (PAYMENT-RESPONSE) header carrying the settlement result. The (PAYMENT-RESPONSE) returns the txId of (paymentGroup[paymentIndex]).

Facilitator HTTP API (GoPlausible)

- (POST /verify) — body ({ x402Version, paymentPayload, paymentRequirements }); returns ({ isValid, invalidReason? }) (some integrations also surface (payer)).
- (POST /settle) — same body; returns ({ success, transaction, network }); HTTP 500 ({ error }) on failure.
- (GET /supported) — returns (kinds[]), (extensions[]), (signers{}). Confirmed live: Algorand signer/fee-payer = (ZMFK20I7ZBD2U27ISERZC4S6LKM6WMFJPZQ4MYNJDZ2VNBNMBA67RA22AA) (same for mainnet and testnet, v1 and v2 kinds). EVM signer (0x136008978ad053942dCDBE759A0903f5d84966fa) (Base (eip155:8453), Base Sepolia (eip155:84532)); Solana signer/fee-payer (8a8fFNfk2AGS7rgVv1BoqPUWnzQuoCrShJV8tSE6RAYi). The facilitator therefore spans EVM, SVM, and AVM at one endpoint.
- No API key required (unlike Coinbase CDP, which requires CDP keys). Rate limits are not documented. The (/docs) page is a JS-rendered SPA whose body was not machine-extractable. A possible Bazaar (/discovery) route on the *hosted* facilitator could not be confirmed live (the (@x402-avm/extensions) Bazaar capability exists as buildable code with

an `onAfterSettle` cataloging hook, but the hosted endpoint was not verified). The reference facilitator can also be self-hosted (env: `AVM_PRIVATE_KEY` base64 64-byte, `ALGOD_SERVER`, default AlgoNode endpoints (`https://mainnet-api.algonode.cloud` / `https://testnet-api.algonode.cloud`)).

Packages and install methods

TypeScript (GoPlausible `@x402-avm/*` scope, recommended for Algorand): `npm install @x402-avm/core @x402-avm/avm`, then middleware `@x402-avm/express` | `@x402-avm/hono` | `@x402-avm/next`, clients `@x402-avm/fetch` | `@x402-avm/axios`, plus `@x402-avm/paywall` (Pera/Defly/Lute paywall UI via `@txnlab/use-wallet`) and `@x402-avm/extensions` (Bazaar + SIWx). Python: `pip install x402-avm[avm]` (extras: `fastapi`, `flask`, `httpx`, `requests`, `extensions`, `all`); import as `from x402...` (not `x402_avm`). As of v2.6, `algosdk` is no longer a dependency—packages use `@algorandfoundation/algokit-utils@10.0.0-alpha.39` internally.

Signer interfaces: `ClientAvmSigner { address: string; signTransactions(txns: Uint8Array[], indexesToSign?: number[]): Promise<(Uint8Array|null)[]> }` (compatible with `@txnlab/use-wallet`); `FacilitatorAvmSigner { getAddresses(); signTransaction(); getAlgodClient(); simulateTransactions(); sendTransactions(); waitForConfirmation() }`. `toClientAvmSigner(base64Key)` and `toFacilitatorAvmSigner(base64Key)` build them; keys are base64 64-byte (32-byte Ed25519 seed + 32-byte pubkey), `address = encode_address(secret_key[32:])`. Exported AVM constants: `ALGORAND_MAINNET_CAIP2`, `ALGORAND_TESTNET_CAIP2`, `ALGORAND_MAINNET_GENESIS_HASH`, `ALGORAND_TESTNET_GENESIS_HASH`, `USDC_TESTNET_ASA_ID`, `V1_ALGORAND_MAINNET`, `V1_ALGORAND_TESTNET`.

Minimal server pattern (Express): create `HTTPFacilitatorClient({ url })`, `new x402ResourceServer(facilitatorClient)`, `registerExactAvmScheme(server)`, then `app.use(paymentMiddleware(routes, server))` with each route declaring `accepts: { scheme: "exact", network, payTo, price }`. Minimal client pattern (Fetch): `new x402Client()`, `registerExactAvmScheme(client, { signer })` (or `client.register("algorand:*", new ExactAvmScheme(avmSigner))`), `const paidFetch = wrapFetchWithPayment(fetch, client)`—the wrapper transparently catches 402, signs, and retries.

Competition context (secondary)

The **Global x402 Challenge** (`https://algorand.co/global-x402-challenge`, rules PDF `https://algorand.co/hubfs/x402%20competition%20official%20Rules.pdf`): prize pool \$100K USD + 500K ALGO. Top 5 cash: \$25K / \$22.5K / \$20K / \$17.5K / \$15K; 500K ALGO split across the top 20 leaderboard. Per the Official Rules PDF, registration June 12–September 1, 2026; final-presentation submission window through September 29, 2026; shortlist September 30–October 8

(finalists notified October 9); finals November 2, 2026; winners by November 12, 2026. (Algorand's public pages instead describe a "mid-July opening, build through early October, finals at Devcon 8 India" timeline—see Caveats.) Eligibility: 18+, not in sanctioned jurisdictions (China, Cuba, Iran, North Korea, Russia, Venezuela, Crimea/Donetsk/Luhansk), English-proficient, not Administrator/Sponsor staff. **Hard requirement:** a paid x402 endpoint deployed and reachable on **Algorand Mainnet** that **uses the GoPlausible facilitator** so volume is tracked on a public leaderboard. Judging (evenly weighted): Volume (USDC processed), Use-case quality (x402 in the core payment flow, not bolted on), Sustained potential, Innovation. Top 50 on the leaderboard are considered for 10 finalist spots presenting live at Devcon 8. Wash trading / repeated self-payments / artificial volume are explicitly disqualifying. Entrants retain all IP; governing law is Delaware.

Ecosystem connective tissue

- **ERC-8004** ("Trustless Agents"): authored by Marco De Rossi (MetaMask), Davide Crapis (Ethereum Foundation), Jordan Ellis (Google), and Erik Reppel (Coinbase); EIP created 2025-08-13. Per Eco.com's support docs, it "went live on Ethereum mainnet" on January 29, 2026. Identity Registry (`0x8004A169FB4a3325136EB29fA0ceB6D2e539a432`) (matches your AgenticPlace deployment), Reputation Registry (`0x8004BAa17C55a88189AE136b182e5fdA19dE9b63`). It defines three registries (Identity, Reputation, Validation); the Identity Registry is an ERC-721 (URIStorage) whose tokenURI points to an agent registration JSON ("agent card") listing endpoints (MCP/A2A/web) and a payment address. Payments are explicitly out of scope, but the spec notes x402 payments can enrich feedback signals. ([GitHub](#))
- **AP2 / A2A-x402**: Google's Agent Payments Protocol (AP2) uses W3C Verifiable Credential "mandates" (Intent/Cart/Payment) for non-repudiable proof of user intent; the A2A x402 extension (<https://github.com/google-agentic-commerce/a2a-x402>), built by Google with Coinbase, Ethereum Foundation, MetaMask) is the crypto-payment path layered on AP2's trust model. GoPlausible has built an AP2-x402 integration extension for A2A on Algorand. MCP is also an x402 transport (paid MCP tools via `@x402/mcp`). ([Algorand](#))
- **Bazaar**: x402's discovery layer ("search engine for agents"); `@x402-avm/extensions/bazaar` lets servers declare `declareDiscoveryExtension({ input, inputSchema, output })` metadata (and `toolName`/`description` for MCP routes) that facilitators catalog after settlement. The discovery extension is chain-agnostic. Also SIWx (CAIP-122 wallet sign-in) extension, in development. Ecosystem explorers include x402scan and various Bazaar dashboards.

Details

Wiring x402 into mindX's API (priority)

mindX is the priority system and already exposes an API and publishes to rage.pythai.net. Gate

the monetizable mindX endpoints behind x402-avm. Because mindX is Python ≥ 3.12 , use the Python SDK: `(pip install "x402-avm[avm,fastapi]")`. Register the AVM scheme `(register_exact_avm_facilitator)` / resource-server middleware), point the `(HTTPFacilitatorClient)` / `(FacilitatorConfig)` at `(https://facilitator.goplausible.xyz)`, set `(network=ALGORAND_MAINNET_CAIP2)`, `(asset="31566704")`, `(extra={"decimals":6})`, and `(payTo)` = a mindX-owned Algorand address that **has opted in to ASA 31566704** (opt-in is mandatory on Algorand or asset transfers are rejected). Price BDI cognitive calls / inference per-request (e.g. `($.01)`–`($.00)`; recall USDC has 6 decimals, so `($.10)` = `(100000)` atomic units). Declare Bazaar discovery metadata so autonomous agents can find mindX inference. Each settled call returns a txId you can log to `rage.pythai.net` as a verifiable receipt. Use the sync variants `(x402ResourceServerSync)`, `(HTTPFacilitatorClientSync)` if mindX runs WSGI/Flask rather than ASGI/FastAPI.

Wiring AgenticPlace (ERC-8004 marketplace)

AgenticPlace already anchors agent identity on Ethereum mainnet via the ERC-8004 Identity Registry at `(0x8004A169FB4a3325136EB29fA0ceB6D2e539a432)`. ERC-8004 is identity/reputation only—payments are deliberately out of scope, which is exactly the gap x402-avm fills. The clean architecture: ERC-8004 NFT identity (Ethereum) + an agent card listing an Algorand `(payTo)` and x402 endpoint metadata → buyers/agents discover via the registry, then pay per call over x402-avm on Algorand → mirror x402 transaction history back as ERC-8004 reputation feedback signals (the spec explicitly anticipates this enrichment). Maintain your canonical chain registry at `(agenticplace.pythai.net/allchain.html)` as the source of truth mapping CAIP-2 IDs (Ethereum `(eip155:1)`, Algorand `(algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkIt8=)`). This keeps identity and settlement on the chains each does best, with CAIP-2 as the lingua franca across both x402 and ERC-8004 (whose Identity Registry aligns with CAIP-10).

Wiring BANKON DAIO payment rails via Parsec

BANKON is the identity/payment/DAIO governance layer being deployed to chain now, with Algorand as your constitutional/payment rail. Parsec (your Algorand-first multi-chain wallet wrapping x402-avm, with the existing `(parsec-x402)` npm module using pure `(node:*)` deps) is the client side: it implements the `(ClientAvmSigner)` interface `((address) + (signTransactions(txns, indexesToSign?)))`. Because v2.6 x402-avm dropped the hard `(algosdk)` dependency, your pure-`(node:*)` constraint is easier to maintain—but verify `(@algorandfoundation/algokit-utils)` transitive deps satisfy your standard, or keep `(parsec-x402)` independent and depend only on `(@x402-avm/core)` types while hand-rolling the msgpack/Ed25519 path. For DAIO treasury flows, the atomic-group composability of the Algorand scheme lets you bundle a governance action (app call) + payment + usage-metering call in a single all-or-nothing group (up to 16 top-level transactions, 256 inner), settling together with no intermediate states—a natural fit for a constitutional payment rail where authorization and settlement must be atomic.

The facilitator's fee-payer co-signs an index-0 `(pay)` self-payment with pooled fee, so mindX agents and BANKON users can pay USDC without holding ALGO. The hosted GoPlausible fee-payer is `(ZMFK20I7...)`; for production sovereignty you may run your own facilitator (set `(AVM_PRIVATE_KEY)`, `(ALGOD_SERVER)`) and become your own fee-payer—but the competition requires the GoPlausible facilitator, so route competition traffic through it. Facilitators offering gasless transactions may refuse to sign a fee-payer txn deemed malicious; such checks can be offloaded to a logic signature.

Recommendations

Stage 1 — Prove the rail (now, days): Stand up one mindX paid endpoint on a non-critical route using `(x402-avm[avm, fastapi])`, `(payTo)` an opted-in mindX mainnet address, facilitator = GoPlausible, price `($.01)` USDC. Drive a real Parsec payment end-to-end; confirm the txId on-chain via AlgoNode/explorer. This simultaneously qualifies for the Challenge leaderboard.

Benchmark to advance: one successful mainnet settle with a verifiable txId.

Stage 2 — Productionize the Parsec client (weeks): Harden `(parsec-x402)` to the `(ClientAvmSigner)` contract, pin it to `(@x402-avm/core)` types only, and audit transitive deps against your pure-`(node:*)` / Apache-2.0 standard. Add a max-price guard (the upstream fetch wrapper defaults to a 0.1 USDC cap—replicate or tighten this) and an `(onBeforePaymentCreation)` policy hook to abort over-budget calls. **Benchmark:** Parsec signs and pays against mainnet with no non-`(node:*)` runtime deps.

Stage 3 — Wire AgenticPlace + BANKON (weeks): Publish ERC-8004 agent cards advertising Algorand x402 endpoints; feed settled x402 txIds back as reputation signals. Add Bazaar discovery metadata to every paid route so agents can auto-discover mindX/AgenticPlace services. For the DAIO, model treasury payments as atomic groups bundling governance + payment + metering. **Benchmark:** an external agent discovers a listed service via Bazaar metadata and pays autonomously.

Stage 4 — Sovereignty (optional): Deploy your own facilitator for non-competition traffic to control pricing/policy and remove the third-party fee-payer dependency. **Trigger:** see thresholds below.

Production-standards checklist: Mainnet-only deployment ✓ (both the scheme and competition require it). Apache-2.0 ✓ (upstream `(@x402/*)` is Apache-2.0; confirm the `(@x402-avm/*)` package licenses before standardizing). For BANKON DAIO Solidity contracts: Foundry as canonical test framework, no admin keys post-deploy, no upgradeable proxies—**note** ERC-8004's reference Identity Registry is an *upgradeable* ERC-721, so if you deploy your own registry variant, reconcile against your no-proxy standard (prefer an immutable redeploy over a proxy). Podman over Docker for facilitator/containerized services. Python ≥3.12 for mindX. USDC ASA `(31566704)`, CAIP-2 `(algorand:wGHE2Pwvdv7S12BL5Fa0P20EGYesN73ktiC1qzkkIt8=)` as canonical.

Thresholds that change the plan: If GoPlausible facilitator rate limits or downtime bite (both undocumented), self-host immediately (Stage 4). If `@algorandfoundation/algokit-utils` pulls non-`node:*` deps that violate Parsec's purity, keep `parsec-x402` on a hand-rolled `msgpack/Ed25519` path depending only on core types. If the competition matters for funding/visibility, front-load Stage 1 and register before the registration window closes (confirm the exact deadline against the live rules).

Caveats

- **Finality figure:** "instant block-level finality" is exact, but the headline average has moved—Algorand's Developer Portal now cites ~2.82-second average block times after the go-algorand v3.21 Dynamic Round Times upgrade (the older ~3.3s figure dates to go-algorand 3.16). Use ~2.8–3.3s as the practical range.
- The literal `price/asset/payTo` in the `https://x402.goplausible.xyz/protected` demo's 402 body could not be retrieved (the endpoint returns 402 but the body was not machine-extractable); curl it directly to capture exact values.
- A live Bazaar `/discovery` route on the *hosted* GoPlausible facilitator could not be confirmed; treat it as unverified until a live GET succeeds. The Bazaar extension itself is real and shippable in your own facilitator.
- **Two npm scopes coexist** (`@x402/*` upstream and `@x402-avm/*` GoPlausible). The Algorand developers page links the upstream `@x402/*` packages, while GoPlausible's own docs use `@x402-avm/*`. Decide which scope you standardize on; they share APIs but version independently (GoPlausible at v2.6.1). For Algorand-first work the `@x402-avm/*` line tracks the AVM scheme most closely; for multichain parity the upstream `@x402/*` line is the foundation reference.
- **Header convention is inconsistent in the spec:** current GoPlausible/Coinbase docs use `X-PAYMENT` (request) and `PAYMENT-RESPONSE` (response), while older spec prose uses `PAYMENT-SIGNATURE`/`PAYMENT-REQUIRED`. Use `X-PAYMENT` for the active GoPlausible AVM implementation. The facilitator endpoints themselves take a JSON body (`paymentPayload` + `paymentRequirements`), not the header.
- **Competition dates conflict across sources:** the Official Rules PDF gives precise dates (registration June 12–September 1, 2026; finals November 2, 2026), while Algorand.co's homepage/press describe a vaguer "mid-July opening, October measurement, Devcon 8 India finals." The rules also state dates "might undergo minor adjustments." Verify against the live Official Rules before relying on any specific date.
- Facilitator/leaderboard data is provided "as-is"; the rules disclaim liability for facilitator outages affecting rankings—another argument for owning your own facilitator path for production-critical (non-competition) traffic.